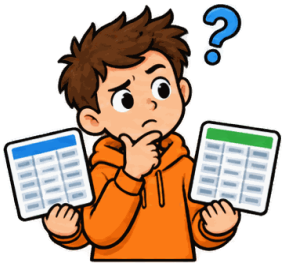
Ticketkontrolle beim Schulfest

Lineare Suche — Schritt für Schritt durch eine Liste

AUSGANGSSITUATION



Die SMV hat **300 Tickets** für das Schulfest verkauft. Jedes Ticket trägt eine Nummer. Mia steht am Einlass mit der Liste der verkauften Nummern – in der Reihenfolge, in der sie verkauft wurden, also unsortiert. Ein Gast zeigt Ticket **2290**. Mia geht die Liste von oben nach unten durch. Der nächste Gast zeigt Ticket **3333**. Diese Nummer wurde nie verkauft – das Ticket ist gefälscht.

Wie viele Nummern muss Mia vergleichen – und bei welchem Gast dauert die Kontrolle länger: beim echten Ticket oder beim gefälschten?

Was tippst du? Kreuze an, bevor du zählst:

☐ beim echten Ticket ☐ beim gefälschten Ticket ☐ gleich lang

Mias Liste – die ersten zwölf verkauften Tickets (gesetzte Annahme)

Index	0	1	2	3	4	5	6	7	8	9	10	11
Nummer	4711	1204	3390	2875	4102	1560	3921	2290	4488	1733	3057	2601

1 Aufgabe 1 — Aufwärmen

Nenne aus dem Gedächtnis den Python-Ausdruck, mit dem du aus der Liste `tickets` die Nummer **3390** erhältst.

SO GEHT'S

So sucht ein Rechner: die lineare Suche

1. Beginne beim **ersten Element** (Index 0).
2. Vergleiche das Element mit dem gesuchten Wert.
3. Gleich? Dann ist der Wert gefunden – merke dir den Index und **höre auf**.
4. Nicht gleich? Gehe zum nächsten Element und vergleiche wieder.
5. Ist die Liste zu Ende und nichts hat gepasst, gilt: **nicht vorhanden**.

Beispiel: Suche nach 3390 in Mias Liste. Vergleich 1: 4711 – nein. Vergleich 2: 1204 – nein. Vergleich 3: 3390 – ja, gefunden bei Index 2. Die Suche brauchte **3 Vergleiche**.

2 Aufgabe 2

Vervollständige das Ablaufprotokoll für die Suche nach Ticket **2290**. Die ersten zwei Schritte sind eingetragen. Trage darunter die Anzahl der Vergleiche und den Index ein.

Vergleich Nr.	Index	Nummer in der Liste	Ist es 2290? (ja / nein)
1	0	4711	nein
2	1	1204	nein
3			
4			
5			
6			
7			
8			
9			
10			

Anzahl der Vergleiche: _____ 2290 steht bei Index: _____

3 Aufgabe 3

Erkläre, warum Mia bei Ticket **3333** erst nach der letzten Nummer der Liste sicher sein kann, dass das Ticket gefälscht ist.

Satzanfänge: „Solange noch eine Nummer übrig ist, …“ · „Erst wenn …, dann …“

MERKE

Bester und schlimmster Fall

Wie viele Vergleiche die lineare Suche braucht, hängt davon ab, **wo der Wert steht**. Steht er ganz vorn, reicht ein Vergleich. Steht er ganz hinten oder fehlt er, muss jeder Eintrag angeschaut werden. Die Anzahl der Einträge nennt man **n**.

bester Fall: 1 Vergleich · schlimmster Fall: n Vergleiche

4 Aufgabe 4

Bestimme die Anzahl der Vergleiche für Mias Ausschnitt, für die ganze Liste und für den Fall, dass die SMV nächstes Jahr doppelt so viele Tickets verkauft.

Fall	Beispiel	n = 12	n = 300	n = 1200
bester Fall	Ticket 4711			
schlimmster Fall	Ticket 3333 (gefälscht)			

5  Aufgabe 5 — an den Rechner

Algorithmen-Werkstatt → Lineare Suche → **Selbst machen**. Dort klickst du die Suche Schritt für Schritt selbst durch – die Website sagt dir sofort, wenn ein Schritt falsch ist.

`https://philipp-byte.github.io/algorithmen-werkstatt/lineare-suche.html`

Dokumentiere zwei Suchläufe mit einer Zufallsliste: die Liste, den gesuchten Wert, den gefundenen Index und die Anzahl der Vergleiche.

Deine Liste	Gesucht	Index (oder „fehlt“)	Vergleiche

INFO

Vom Ablauf zum Python-Code

`range(len(liste))` liefert alle Indizes von 0 bis zum letzten – bei zwölf Einträgen also 0 bis 11. Mit `liste[i]` greifst du auf das Element mit dem Index `i` zu.

`return` **beendet die Funktion sofort**. Alles, was danach kommt, wird nicht mehr ausgeführt – die Schleife läuft also nicht weiter, sobald etwas zurückgegeben wurde.

Für „nicht gefunden“ gibt man ein **Signal** zurück, das nie ein echter Index sein kann. Üblich ist die Zahl `-1`.

6 Aufgabe 6

Implementiere die fehlenden Teile der Funktion. Sie soll den Index des gesuchten Werts zurückgeben – oder −1, wenn er fehlt.

```
1  def lineare_suche(liste, gesucht):
2      for i in range( _________ ):
3          if _________ == gesucht:
4              return _________
5      return _________
6
7  tickets = [4711, 1204, 3390, 2875, 4102, 1560, 3921, 2290, 4488, 1733, 3057, 2601]
8  print(lineare_suche(tickets, 2290))
```

Wenn du nicht weiterkommst: In welcher Zeile ist die Liste zu Ende gelaufen, ohne dass etwas gefunden wurde?

7 Aufgabe 7 — am Rechner

Teste deine Funktion in der Werkstatt unter **Code schreiben** (oder in Thonny) mit Mias Liste. Trage die Ausgaben ein und kreuze an, wenn alle Tests bestanden sind.

Ausgabe für 2290: _____ Ausgabe für 3333: _____ ☐ alle Tests bestanden



Zähle nicht die Zeilen der Tabelle, sondern die **Vergleiche**: Auch die letzte Nummer, die nicht passt, hat Mia angeschaut – das ist ein Vergleich.

8 Aufgabe 8 — Mias Frage

Begründe mit deinen Zahlen aus Aufgabe 2 und 4, bei welchem Gast Mias Kontrolle länger dauert – und was das für den Einlass mit 300 Tickets bedeutet. Vergleiche am Ende mit deinem Tipp vom Anfang.

Satzanfänge: „Beim echten Ticket braucht Mia …, beim gefälschten …“ · „Das liegt daran, dass …“ · „Für 300 Tickets heißt das …“

9 Aufgabe 9 — Vertiefung

Erweitere die Funktion aus Aufgabe 6 so, dass sie mitzählt, wie viele Vergleiche sie gemacht hat, und beides zurückgibt: `return index, vergleiche`. Schreibe die vollständige Funktion auf.

10 Aufgabe 10 — Zusatz

Entwickle ein Vorgehen, mit dem die SMV nach dem Fest herausfindet, welche verkauften Tickets **nicht eingelöst** wurden. Mia hat dafür eine zweite, ebenfalls unsortierte Liste mit allen am Einlass gescannten Nummern. Halte die Schritte in Stichpunkten fest und gib an, wie viele Vergleiche dein Vorgehen bei 300 verkauften und 300 gescannten Tickets höchstens braucht.

PRÜFE DICH SELBST

- ☐ Steht in deinem Protokoll aus Aufgabe 2 genau einmal „ja“ – in der letzten ausgefüllten Zeile?
- ☐ Ist dein Wert für n = 1200 viermal so groß wie der für n = 300?
- ☐ Gibt deine Funktion für ein Ticket, das fehlt, die Zahl −1 zurück – und nicht `None`?